ChatGPT 5.2

now give a search leetcode question

Nice, classic interview territory 😄
Here's a **clean LeetCode-style *search* question** that shows up *all the time*:

---

## 🔍 Search in Rotated Sorted Array

### 🧠 Problem Statement

You are given an integer array `nums` that was originally sorted in **ascending order**, but then **rotated at some unknown pivot**.

For example:

```
[0,1,2,4,5,6,7]  → rotated →  [4,5,6,7,0,1,2]
```

You are also given an integer `target`.

**Return the index of `target` if it exists in `nums`, otherwise return `-1`.**

---

### 📌 Constraints

- `1 <= nums.length <= 10^4`
- All elements in `nums` are **unique**
- `nums` is rotated **exactly once**
- Your algorithm must run in **O(log n)** time

---

### ✨ Example 1

```
makefile
```

```
Input: nums = [4,5,6,7,0,1,2], target = 0
Output: 4
```

### ✨ Example 2

makefile Copy code

```
Input: nums = [4,5,6,7,0,1,2], target = 3
Output: -1
```

---

### 🎯 Interview Signal

What the interviewer is testing:

- Binary search adaptation
- Reasoning about **sorted halves**
- Handling edge conditions cleanly

---

### 💡 Hint (not solution)

At every step:

- One half of the array is **always sorted**
- Decide:
  - Is the `target` in the sorted half?
  - Or in the other half?

---

If you want:

- ✅ full Python solution (clean + interview-friendly)
- ✅ step-by-step dry run
- ✅ edge cases interviewers love
- ✅ or a follow-up variant (duplicates / minimum element / pivot search)

Just say the word 👌

brute force approach

Perfect — let's keep this **interview-clear and no fancy tricks** 👍
Here's the **brute force approach** for **Search in Rotated Sorted Array**.

---

## 🔍 Brute Force Approach

### 🧠 Idea

Ignore the fact that the array is sorted or rotated.

Just:

- Scan the array **linearly**
- Compare each element with `target`
- Return the index if found

If you reach the end → return `-1`

---

### ⏱️ Time & Space Complexity

- **Time:** `O(n)`
- **Space:** `O(1)`

  Interview note:
  This **does NOT meet the O(log n)** requirement, but it's a **valid baseline** and good to mention before optimizing.

---

## 🧩 Python Implementation (Readable + Testable)

python Copy code

```python
def search_brute_force(nums, target):
    """
```

```
    Brute force search for target in nums.
    Returns index if found, otherwise -1.
    """
    for i in range(len(nums)):
        if nums[i] == target:
            return i
    return -1


def main():
    nums = [4, 5, 6, 7, 0, 1, 2]
    target = 0

    result = search_brute_force(nums, target)
    print("Index:", result)


if __name__ == "__main__":
    main()
```

---

## 🧪 Example Walkthrough

ini

Copy code

```
nums = [4, 5, 6, 7, 0, 1, 2]
target = 0

i=0 → 4 ≠ 0
i=1 → 5 ≠ 0
i=2 → 6 ≠ 0
i=3 → 7 ≠ 0
i=4 → 0 == target ✅ → return 4
```

---

## 🎤 What to Say in an Interview

"The brute-force approach scans the array linearly in O(n) time.
Since the array is rotated but still contains unique elements, we can do better using a

modified binary search to achieve O(log n).”

---

Next step?

- 🚀 Optimized **binary search solution**
- 🧠 Intuition first, code second
- 🧪 Edge cases (single element, not rotated, target missing)

Your call 😄

All these: Optimized binary search solution

🧠 Intuition first, code second

🧪 Edge cases (single element, not rotated, target missing)

## 🧠 Intuition (modified binary search)

Even though the array is rotated, **at least one half is still sorted** at every step.

With indices `l..mid..r`:

1. Compute `mid`.
2. If `nums[mid] == target` → done.
3. Decide which half is sorted:
   - If `nums[l] <= nums[mid]` → **left half is sorted**
   - Else → **right half is sorted**
4. Check if `target` lies inside the sorted half’s value range:
   - If yes → binary search inside that half
   - If no → go to the other half

This keeps the search **logarithmic**.

---

## ✅ Optimized Python (clean + interview-friendly)

python Copy code

```python
def search_rotated(nums, target):
    """
    Search target in a rotated sorted array with unique elements.
    Returns index if found, otherwise -1.
    Time: O(log n), Space: O(1)
    """
    l, r = 0, len(nums) - 1

    while l <= r:
        mid = (l + r) // 2

        if nums[mid] == target:
            return mid

        # Left half is sorted
        if nums[l] <= nums[mid]:
            if nums[l] <= target < nums[mid]:
                r = mid - 1
            else:
                l = mid + 1
        # Right half is sorted
        else:
            if nums[mid] < target <= nums[r]:
                l = mid + 1
            else:
                r = mid - 1

    return -1


def run_tests():
    tests = [
        # Standard rotated
        ([4, 5, 6, 7, 0, 1, 2], 0, 4),
        ([4, 5, 6, 7, 0, 1, 2], 3, -1),

        # Single element
```

```
        ([1], 1, 0),
        ([1], 0, -1),

        # Not rotated (already sorted)
        ([1, 2, 3, 4, 5], 4, 3),
        ([1, 2, 3, 4, 5], 6, -1),

        # Small rotations
        ([3, 1, 2], 1, 1),
        ([3, 1, 2], 3, 0),

        # Edge: two elements rotated
        ([2, 1], 1, 1),
        ([2, 1], 2, 0),
    ]

    for nums, target, expected in tests:
        got = search_rotated(nums, target)
        assert got == expected, f"FAIL nums={nums} target={target} expecte
    print("All tests passed!")


def main():
    nums = [4, 5, 6, 7, 0, 1, 2]
    target = 0
    print("Index:", search_rotated(nums, target))
    run_tests()


if __name__ == "__main__":
    main()
```

---

## 🧪 Edge cases (what to say + why they work)

### 1) Single element

- `nums = [1]`
- The loop runs once, either matches or returns `-1`.

- No special-case needed.

### 2) Not rotated

- `nums = [1,2,3,4,5]`
- Condition `nums[l] <= nums[mid]` is always true → behaves like normal binary search.

### 3) Target missing

- Eventually `l` moves past `r` → returns `-1`.
- No infinite loops because each iteration shrinks the search space.

---

If you want a tougher follow-up: **“Search in Rotated Sorted Array II”** (duplicates) — the logic needs one extra trick.